

Lecture review

Polymorphisms means that a call to a member function will cause a different function to be executed depending on the type of object that invokes the function. Typically, polymorphism occurs when there is a hierarchy of classes and they are related by inheritance.

Real world example

- A real life example of polymorphism is that a person at the same time can have different characteristics. A man at the same time is a father, a husband, an employee, so the same person possesses different behavior in different situations. This is called as polymorphism.

TYPES OF POLYMORPHISM

In C++ polymorphism is mainly divided into two types:

- Compile time Polymorphism
- Runtime Polymorphism

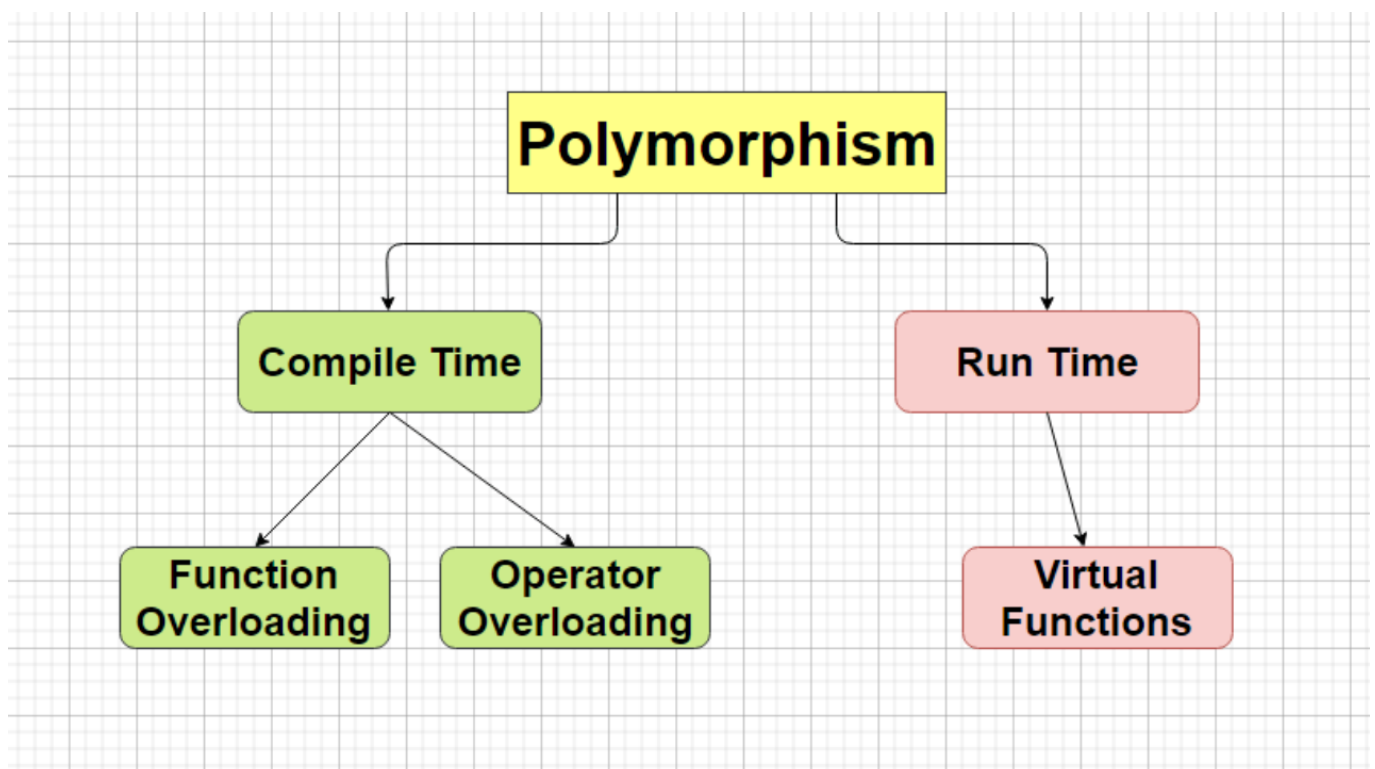


Figure 1: Types of Polymorphism

Compile time Polymorphism

This type of polymorphism is achieved by function overloading or operator overloading.

Function overloading When there are multiple functions with same name but different parameters then these functions are said to be overloaded. Functions can be overloaded by a change in the number of arguments or/and change in the type of arguments.

```
// C++ program for function overloading
#include <bits/stdc++.h>

using namespace std;
class Geeks {
public:

    // function with 1 int parameter
    void func(int x) {
        cout << "value of x is " << x << endl;
    }

    // function with same name but 1 double parameter
    void func(double x) {
        cout << "value of x is " << x << endl;
    }

    // function with same name and 2 int parameters
    void func(int x, int y) {
        cout << "value of x and y is " << x << ", " << y << endl;
    }
};

int main() {

    Geeks obj1;

    // Which function is called will depend on the parameters passed
    // The first 'func' is called
    obj1.func(7);

    // The second 'func' is called
    obj1.func(9.132);

    // The third 'func' is called
    obj1.func(85, 64);
    return 0;
}
```

Sample Run:

```
value of x is 7
value of x is 9.132
value of x and y is 85, 64
```

In the above example, a single function named func acts differently in three different situations which is the property of polymorphism.

Operator Overloading

In C++, the purpose of operator overloading for an operator is to specify more than one meaning in one scope. It gives special meaning of an operator for a user-defined data type. You can redefine most of the C++ operators with the help of operator overloading. To perform multiple operations using one operator, you can also use operator overloading. Syntax of operator overloading:

To overload a C++ operator, you should define a special function inside the Class as follows:

```
class class_name
{
    ... ..
    public
        return_type operator symbol (argument(s))
        {
            ... ..
        }
    ... ..
};
```

Here is an explanation for the above syntax:

- The return_type is the return type for the function.
- Next, you mention the operator keyword.
- The symbol denotes the operator symbol to be overloaded. For example, +, -, <, ++.
- The argument(s) can be passed to the operator function in the same way as functions.

Example:

```
#include <iostream>
using namespace std;

class TestClass {
private:
    int count;

public:
    TestClass() {
        count = 5;
    }

    // Prefix
    void operator--() {
        count = count - 1;
    }

    // Postfix
    void operator--(int) {
```

```

        count = count - 1;
    }

    void Display() {
        cout << "Count: " << count << endl;
    }
};

int main() {
    TestClass tc;
    --tc;
    tc.Display();
    tc--;
    tc.Display();
    return 0;
}

```

output:
Count: 4
Count: 3

C++ Operators that can and cannot be Overloaded

Unary Operators Overloading

The unary operators operate on a single operand and following are the examples of Unary operators.

- The increment (++) and decrement (-) operators.
- The unary minus (-) operator.
- The logical not (!) operator.

The unary operators operate on the object for which they were called and normally, this operator appears on the left side of the object, as in !obj, -obj, and ++obj but sometime they can be used as postfix as well like obj++ or obj-. Following example explain how minus (-) operator can be overloaded for prefix as well as postfix usage.

```

#include <iostream>
using namespace std;

class Distance {
private:
    int feet;           // 0 to infinite
    int inches;         // 0 to 12

public:
    // required constructors
    Distance() {
        feet = 0;
        inches = 0;
    }
    Distance(int f, int i) {

```

Operator	Description / Overloadability
Operators That CAN Be Overloaded	
Arithmetic	+, -, *, /, % (Basic arithmetic operations)
Bitwise	^, &, , ~ (Bitwise XOR, AND, OR, NOT)
Assignment	=, +=, -=, *=, /=, %=, ^=, &=, =, <=, >=
Relational	==, !=, <, >, <=, >= (Comparison operators)
Logical	&&, , ! (Logical AND, OR, NOT)
Increment/Decrement	++, - (Prefix and postfix)
Pointer and Member Access	->, * (Pointer dereference and member access)
Stream	<<, >> (Used in cout and cin)
Function Call	() (Allows an object to be called like a function)
Indexing	[] (Array-style indexing)
Dynamic Memory	new, new[], delete, delete[] (Custom memory management)
Operators That CANNOT Be Overloaded	
Scope Resolution	:: (Used for accessing global or class members)
Pointer-to-Member	.* (Used for member pointer access)
Member Access	. (Cannot be directly overloaded, but -> can)
Size Query	sizeof (Evaluated at compile-time)
Type Identification	typeid (Used for runtime type checking)
Alignment Query	alignof (Checks memory alignment)
Type Casting	static_cast, dynamic_cast, const_cast, reinterpret_cast (Used for explicit type conversions)

Table 1: Overloadable and Non-Overloadable Operators in C++

```

    feet = f;
    inches = i;
}

// method to display distance
void displayDistance() {
    cout << "F: " << feet << " I:" << inches << endl;
}

// overloaded minus (-) operator
Distance operator- () {
    feet = -feet;
    inches = -inches;
    return Distance(feet, inches);
}
};

int main() {

```

```

Distance D1(11, 10), D2(-5, 11);

-D1;                // apply negation
D1.displayDistance(); // display D1

-D2;                // apply negation
D2.displayDistance(); // display D2

return 0;
}

```

Sample output:

F: -11 I: -10

F: 5 I: -11

The increment (++) and decrement (-) operators are two important unary operators available in C++. Following example explain how increment (++) operator can be overloaded for prefix as well as postfix usage. Similar way, you can overload operator (-).

```

#include <iostream>
using namespace std;

class Time {
private:
    int hours;           // 0 to 23
    int minutes;         // 0 to 59

public:
    // required constructors
    Time() {
        hours = 0;
        minutes = 0;
    }
    Time(int h, int m) {
        hours = h;
        minutes = m;
    }

    // method to display time
    void displayTime() {
        cout << "H: " << hours << " M:" << minutes << endl;
    }

    // overloaded prefix ++ operator
    Time operator++ () {
        ++minutes;        // increment this object
        if(minutes >= 60) {
            ++hours;
            minutes -= 60;
        }
        return Time(hours, minutes);
    }
}

```

```

// overloaded postfix ++ operator
Time operator++( int ) {

    // save the original value
    Time T(hours, minutes);

    // increment this object
    ++minutes;

    if(minutes >= 60) {
        ++hours;
        minutes -= 60;
    }

    // return old original value
    return T;
}

};

int main() {
    Time T1(11, 59), T2(10,40);

    ++T1;                // increment T1
    T1.displayTime();     // display T1
    ++T1;                // increment T1 again
    T1.displayTime();     // display T1

    T2++;                // increment T2
    T2.displayTime();     // display T2
    T2++;                // increment T2 again
    T2.displayTime();     // display T2
    return 0;
}

```

When the above code is compiled and executed, it produces the following result –

```

H: 12 M:0
H: 12 M:1
H: 10 M:41
H: 10 M:42

```

Figure 2: Sample output

Binary Operators Overloading

The binary operators take two arguments and following are the examples of Binary operators. You use binary operators very frequently like addition (+) operator, subtraction (-) operator and division (/) operator. Following example explains how addition (+) operator can be overloaded. Similar way, you can overload subtraction (-) and division (/) operators.

```
#include <iostream>
using namespace std;

class Box {
    double length;    // Length of a box
    double breadth;    // Breadth of a box
    double height;    // Height of a box

public:

    double getVolume(void) {
        return length * breadth * height;
    }

    void setLength( double len ) {
        length = len;
    }

    void setBreadth( double bre ) {
        breadth = bre;
    }

    void setHeight( double hei ) {
        height = hei;
    }

    // Overload + operator to add two Box objects.
    Box operator+(const Box& b) {
        Box box;
        box.length = this->length + b.length;
        box.breadth = this->breadth + b.breadth;
        box.height = this->height + b.height;
        return box;
    }
};

// Main function for the program
int main() {
    Box Box1;           // Declare Box1 of type Box
    Box Box2;           // Declare Box2 of type Box
    Box Box3;           // Declare Box3 of type Box
    double volume = 0.0; // Store the volume of a box here

    // box 1 specification
    Box1.setLength(6.0);
    Box1.setBreadth(7.0);
    Box1.setHeight(5.0);

    // box 2 specification
    Box2.setLength(12.0);
    Box2.setBreadth(13.0);
    Box2.setHeight(10.0);
```



```

// volume of box 1
volume = Box1.getVolume();
cout << "Volume of Box1 : " << volume <<endl;

// volume of box 2
volume = Box2.getVolume();
cout << "Volume of Box2 : " << volume <<endl;

// Add two object as follows:
Box3 = Box1 + Box2;

// volume of box 3
volume = Box3.getVolume();
cout << "Volume of Box3 : " << volume <<endl;

    return 0;
}

/*Sample output*/
Volume of Box1-----210
Volume of Box2-----1560
Volume of Box3-----5400

```

Relational Operators Overloading

There are various relational operators supported by C++ language like (<, >, <=, >=, ==, etc.) which can be used to compare C++ built-in data types. You can overload any of these operators, which can be used to compare the objects of a class. Following example explains how a < operator can be overloaded and similar way you can overload other relational operators.

```

#include <iostream>
using namespace std;

class Distance {
    private:
        int feet;           // 0 to infinite
        int inches;         // 0 to 12

    public:
        // required constructors
        Distance() {
            feet = 0;
            inches = 0;
        }
        Distance(int f, int i) {
            feet = f;
            inches = i;
        }

        // method to display distance
        void displayDistance() {

```

```

        cout << "F: " << feet << " I:" << inches <<endl;
    }

    // overloaded < operator
    bool operator <(const Distance& d) {
        if(feet < d.feet) {
            return true;
        }
        if(feet == d.feet && inches < d.inches) {
            return true;
        }

        return false;
    }
};

int main() {
    Distance D1(11, 10), D2(5, 11);

    if( D1 < D2 ) {
        cout << "D1 is less than D2 " << endl;
    } else {
        cout << "D2 is less than D1 " << endl;
    }

    return 0;
}

```

Assignment Operator Overloading

You can overload the assignment operator (=) just as you can other operators and it can be used to create an object just like the copy constructor. Following example explains how an assignment operator can be overloaded.

```

#include <iostream>
using namespace std;

class Distance {
    private:
        int feet;           // 0 to infinite
        int inches;         // 0 to 12

    public:
        // required constructors
        Distance() {
            feet = 0;
            inches = 0;
        }
        Distance(int f, int i) {
            feet = f;
            inches = i;
        }

```

```

    }
    void operator = (const Distance &D ) {
        feet = D.feet;
        inches = D.inches;
    }

    // method to display distance
    void displayDistance() {
        cout << "F: " << feet << " I:" << inches << endl;
    }
};

int main() {
    Distance D1(11, 10), D2(5, 11);

    cout << "First Distance : ";
    D1.displayDistance();
    cout << "Second Distance : ";
    D2.displayDistance();

    // use assignment operator
    D1 = D2;
    cout << "First Distance : ";
    D1.displayDistance();

    return 0;
}

```

Function Call() Operator Overloading

The function call operator () can be overloaded for objects of class type. When you overload () , you are not creating a new way to call a function. Rather, you are creating an operator function that can be passed an arbitrary number of parameters. Following example explains how a function call operator () can be overloaded.

```

#include <iostream>
using namespace std;

class Distance {
private:
    int feet;           // 0 to infinite
    int inches;         // 0 to 12

public:
    // required constructors
    Distance() {
        feet = 0;
        inches = 0;
    }
    Distance(int f, int i) {
        feet = f;
        inches = i;
    }
}

```

```

    }

    // overload function call
    Distance operator()(int a, int b, int c) {
        Distance D;

        // just put random calculation
        D.feet = a + c + 10;
        D.inches = b + c + 100 ;
        return D;
    }

    // method to display distance
    void displayDistance() {
        cout << "F: " << feet << " I:" << inches << endl;
    }
};

int main() {
    Distance D1(11, 10), D2;

    cout << "First Distance : ";
    D1.displayDistance();

    D2 = D1(10, 10, 10); // invoke operator()
    cout << "Second Distance :";
    D2.displayDistance();

    return 0;
}

```

Sample output

```

First  Distance F11 I:10
Second Distance F30 I:120

```

Things to remember for Operator Overloading

Here are rules for Operator Overloading:

- At least one of the operands in overloaded operators must be user-defined, which means we cannot overload the minus operator to work with one integer and one double. However, you could overload the minus operator to work with an integer and a mystring.
- We can only overload the operators that exist and cannot create new operators or rename existing operators.
- You can make the operator overloading function a friend function if it needs to access the private and protected class members.
- Some operators cannot be overloaded using a friend function. However, such operators can be overloaded using member function. Eg. Assignment overloading operator = and function call() operator etc.

Run time Polymorphism

This type of polymorphism is achieved by Function Overriding.

Function Overriding

Function overriding is a feature that allows us to have a same function in child class which is already present in the parent class.

- A child class inherits the data members and member functions of parent class, but when you want to override a functionality in the child class then you can use function overriding. It is like creating a new version of an old function, in the child class.
- To override a function you must have the same signature in the child class.

Syntax for Function Overriding:

```
public class Parent{
access_modifier:
return_type method_name(){
};

public class child : public Parent {
access_modifier:
return_type method_name(){
};
```

Example Code for Function Overriding:

```
#include <iostream>
using namespace std;
class BaseClass {
public:
    void disp(){
        cout<<"Function of Parent Class";
    }
};
class DerivedClass: public BaseClass{
public:
    void disp() {
        cout<<"Function of Child Class";
    }
};
int main() {
    DerivedClass obj = DerivedClass();
    obj.disp();
    return 0;
}
```

Sample Run:

Function of Child Class

```

#include <iostream>
using namespace std;

class Distance {
private:
    int feet; // Feet part of distance
    int inches; // Inches part of distance

public:
    // Constructor
    Distance(int f = 0, int i = 0) {
        feet = f;
        inches = i;
    }

    // Display function
    void displayDistance() {
        cout << "Feet: " << feet << " Inches: " << inches << endl;
    }

    // Prefix decrement (--obj)
    Distance operator--() {
        feet /= 2;
        inches /= 2;
        return *this; // Return modified object
    }

    // Postfix decrement (obj--)
    Distance operator--(int) {
        Distance temp = *this; // Store current state
        feet /= 2;
        inches /= 2;
        return temp; // Return original state before decrement
    }
};

int main() {
    Distance D1(8, 10), D2(16, 20);

    cout << "Before Prefix --: ";
    D1.displayDistance();
    --D1; // Prefix: Multiplies feet and inches by 2
    cout << "After Prefix --: ";
    D1.displayDistance();

    cout << "\nBefore Postfix --: ";
    D2.displayDistance();
    D2--; // Postfix: Divides feet and inches by 2
    cout << "After Postfix --: ";
    D2.displayDistance();

    return 0;
}

```

}

Lab exercises

Exercise 1

Write a program to overload decrement operator `-` in such a way that when it is used as a prefix, it multiplies a number by 4 and when it is used as a postfix then it divides the number by 4.

Exercise 2

Write a program that will apply the concept of operator overloading on `+` operator to add the areas of `shape1` and `shape2`. Name of class is “shape” while `shape1` and `shape2` are the objects of class shape. Use the same `Area()` function for both objects.

Exercise 3

You are assigned a task in IBA sports day which is to find out the area of football ground, cricket ground, and a place for robot playing. The mentioned places are square, rectangle and circular in shape. You are given only one side length for football ground which can be user input by using constructor only and is private. Then there is a constraint that the width of cricket ground is equal to football side but the length is twice, while the robotic play ground radius is equal to the given football side. There is another constraint that you can only use the football given side for finding out the area of cricket ground, and robotic ground along with football ground.

Exercise 4

A software company is developing a **Payment Processing System** that supports multiple payment methods. You are required to implement the program according to the class diagram shown below.

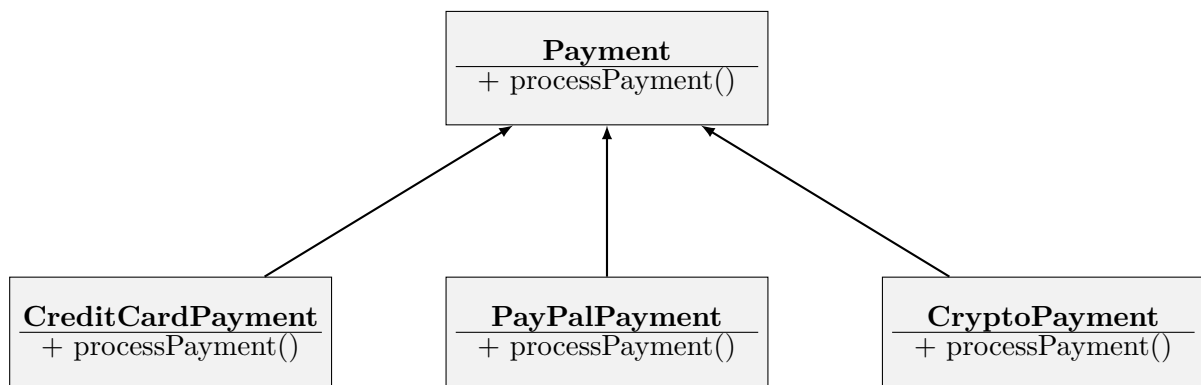


Figure 3: Class Diagram

Note the following:

- The `processPayment()` function in the `Payment` class displays the text:
"Processing a generic payment."
- The `processPayment()` function in the `CreditCardPayment` class displays the text:
"Processing payment via Credit Card."
- The `processPayment()` function in the `PayPalPayment` class displays the text:
"Processing payment via PayPal."
- The `processPayment()` function in the `CryptoPayment` class displays the text:
"Processing payment via Cryptocurrency."

- In the `main()` function, create objects for each of the classes and display the text for each of them.

Exercise 5

You are required to implement a program that allows students to calculate the area/volume of different shapes based on user input. The program will create an object dynamically at runtime based on the user's input: if the user enters two sides (length, width), a **Rectangle** object will be created; if one side (radius) is given, a **Circle** object will be created; and if three sides (a, b, c) are provided, a **Triangle** object will be created.

The area or volume will be calculated according to the shape:

- For a **Rectangle**:
 $Area = Length \times Width$
- For a **Circle**:
 $Area = \pi \times Radius^2$
- For a **Triangle**:
 $Area = \sqrt{s(s-a)(s-b)(s-c)}$, where $s = \frac{a+b+c}{2}$ (Heron's formula)

You are also required to display the appropriate result for each calculation.

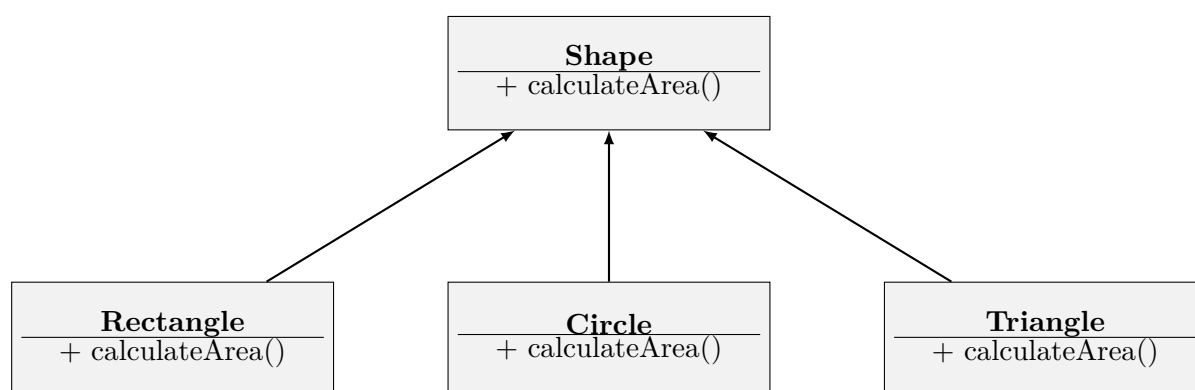


Figure 4: Class Diagram for Shape Calculation

- The `calculateArea()` function in the **Shape** class displays:
"Calculating area of a shape."
- The `calculateArea()` function in the **Rectangle** class displays the calculated area:
"Area of Rectangle: $Length \times Width = \{calculated_area\}$ "
- The `calculateArea()` function in the **Circle** class displays the calculated area:
"Area of Circle: $\pi \times Radius^2 = \{calculated_area\}$ "
- The `calculateArea()` function in the **Triangle** class displays the calculated area:
"Area of Triangle (Heron's formula): $\sqrt{s(s-a)(s-b)(s-c)} = \{calculated_area\}$ "
- In the `main()` function, prompt the user for input. Based on the input (number of sides), dynamically create an object for the corresponding shape, and then call the appropriate method to calculate and display the area.

Exercise 6

A company has a team of superheroes working in different departments, and its your task to design an

automated bonus calculator to reward their efforts. The company wants to calculate the bonuses for their hard-working employees, and they need a program that can do this with a *snap of your fingers*. The goal is to develop a system where every hero gets a well-deserved bonus. Your mission: Create a program with the following steps!

- **Create a Base Class: `Person`** This is the foundation of your program. The base class will have:
 - A data member to store the Employees ID
 - Member functions `getData` and `displayData` to handle input and output.
- **Create Derived Classes: `Admin` & `Accounts`** These two classes will inherit from `Person` and will also have:
 - `getData`, `displayData` (inherited from `Person`), and a new function `bonus` to calculate each employees bonus.
- **The Bonus Formula!** Every employee gets an **annual bonus of 5% of their monthly income** (lets reward those superheroes!). The `Admin` and `Accounts` classes will store the employees name and monthly income.
- **Display the Info** For each employee, print their:
 - Employee ID
 - Name
 - Monthly Income
 - Bonus amount they received.

Extra Features: If you want to make your program even more magical, try adding:

- Input validation to ensure that the employee's name and income are valid.
- A system to handle multiple bonuses, like "Best Employee of the Month" .
- An interactive interface where employees can input their information using a fun command-line interface!

Exercise 7

Your next task is to create a magical tool that will allow students at a school to perform **multiplication operations** effortlessly. You're going to make it super easy by writing four functions that have the same name. **Yes, you heard that right!** The power of overloading is in your hands!

Here are your four *wizardly* functions:

- **Multiply Two Integers** Create a function that takes two integers and returns the result of multiplying them.
- **Multiply Three Integers** Create a second function that takes three integers and multiplies them together.
- **Multiply Two Decimals** This one will handle decimal values! Create a function that multiplies two decimal numbers and returns the result.

- **Multiply Three Decimals** Finally, make a function that multiplies three decimal values and returns the product.

Display the results for all four functions to the user.

Exercise 8

A vehicle rental company operates in a large city, offering cars, bikes, and trucks for rent. The company has a fleet of different vehicle types, each with its own rental rates and features. Your task is to design and implement a system that automates the vehicle rental process and calculates the total rental cost for customers, applying different pricing logic based on vehicle type, rental duration, and payment method.

The company wants to implement the following system, which will involve various **object-oriented programming (OOP)** concepts such as **polymorphism**, **function overloading**, **operator overloading**, and **function overriding**. You will need to design the classes and implement the necessary operations.

Requirements:

1. Vehicle Class (Base Class):

- **Attributes:**
 - `rentalPricePerDay` (a double): The daily rental price for the vehicle.
 - `vehicleType` (a string): The type of vehicle (e.g., "Car", "Bike", "Truck").
- **Methods:**
 - `rent(int days)` (pure virtual): Calculates the total rental cost for the given number of days (to be overridden in derived classes).
 - `displayInfo()` (overridden): Displays basic information about the vehicle, including its type and rental price.

• 2. Car Class (Derived from Vehicle):

- **Attributes:** Inherits `rentalPricePerDay` and `vehicleType`, but should have a specific `airConditioning` feature.
- **Methods:**
 - * `rent(int days)` (overridden): Calculates rental cost with a 10% discount if the rental duration is over 7 days.
 - * `displayInfo()` (overridden): Displays information specific to a car (e.g., air conditioning, number of seats).

• 3. Bike Class (Derived from Vehicle):

- **Attributes:** Inherits `rentalPricePerDay` and `vehicleType`, but should have a specific `helmetIncluded` feature.
- **Methods:**
 - * `rent(int days)` (overridden): Calculates rental cost with a 5% discount if the rental duration is more than 3 days.
 - * `displayInfo()` (overridden): Displays information specific to a bike (e.g., helmet included, fuel type).

- **4. Truck Class (Derived from Vehicle):**
 - **Attributes:** Inherits `rentalPricePerDay` and `vehicleType`, but should have a specific `cargoCapacity` feature.
 - **Methods:**
 - * `rent(int days)` (overridden): Calculates rental cost without any discounts, but adds a surcharge of 20% for rentals over 5 days due to high demand.
 - * `displayInfo()` (overridden): Displays information specific to a truck (e.g., cargo capacity, number of wheels).
- **5. Function Overloading:**
 - Implement multiple `rent` functions in the base class for different payment methods. Overload the function to accept either a string for the payment method or a double for the payment amount.
- **6. Operator Overloading:**
 - Overload the `+` operator to compare rental costs between two vehicles. This will allow customers to compare which vehicle is cheaper.

Instructions

You need to implement the system with the following operations:

1. **Rental Calculation:**
 - Based on the type of vehicle (Car, Bike, Truck), the rental cost needs to be calculated for a given number of days. Discounts and surcharges must be applied where appropriate.
2. **Comparing Vehicles:**
 - You need to compare the rental costs of two different vehicles using the overloaded `+` operator. This will help the customer decide which vehicle is cheaper.
3. **Multi-payment Method:**
 - Allow the system to handle multiple types of payment methods: cash, credit card, and online payment. Function overloading must be used for this. When a customer rents a vehicle, they can choose to pay by cash, credit card, or online, and the system should process the payment accordingly.
4. **Displaying Vehicle Information:**
 - When a customer rents a vehicle, the system should display detailed information about the vehicle, including whether it has air conditioning, a helmet, or cargo capacity, depending on the type of vehicle.

Example Scenario:

The company wants to process two rental requests:

- **Customer 1:** Wants to rent a car for 10 days and chooses to pay using a credit card.

- **Customer 2:** Wants to rent a bike for 4 days and will pay using cash.

You will need to:

1. Calculate the rental price for each customer.
2. Display the specific information about each vehicle (car and bike).
3. Compare the rental costs of the car and bike using the overloaded operator.

Sample Output:

Customer 1 (Car Rental):

Vehicle Type: Car

Rental Price per Day: \$50.0

Air Conditioning: Yes

Seats: 4

Total Rental for 10 Days: \$450.0

Payment Method: Credit Card

Customer 2 (Bike Rental):

Vehicle Type: Bike

Rental Price per Day: \$20.0

Helmet Included: Yes

Fuel Type: Gas

Total Rental for 4 Days: \$76.0

Payment Method: Cash

Comparison of Rental Costs:

The car rental is more expensive than the bike rental.

- Use **polymorphism** to handle vehicle rental through a common interface (the `rent()` method).
- Implement **operator overloading** to compare rental costs.
- **Function overloading** should allow for different types of payments in the `rent()` method.
- Use **function overriding** to customize behavior in derived classes for each vehicle type.

